

University of British Columbia

ELEC291: Electrical Engineering Design Studio I

Winter 2025-26

Instructor: Dr. Jesús Calviño-Fraga

Section 201

Project 2: Field Tracking Robot

Group #: C12

Student name
Daniel Gu
How Yee Chaw
Jacky Zou
Yichen Liu
Qijun Zhou

Date of Submission: April 10, 2026

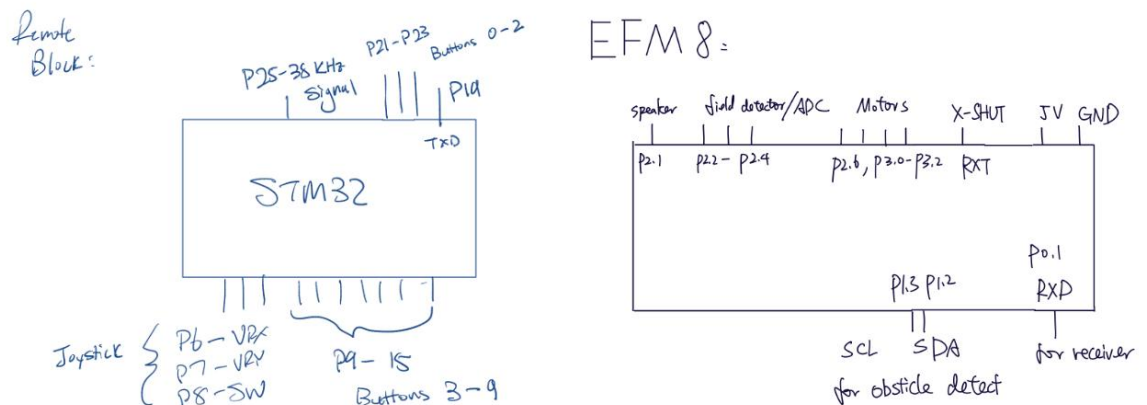
Table of Contents

I.	Introduction	p.2
II.	Investigation	
	A. Idea Generation	p.2
	B. Investigation Design	p.3
	C. Data Collection	p.4
	D. Data Synthesis	p.4
	E. Analysis of Results	p.5
III.	Design	
	A. Use of Process	p.6
	B. Need and Constraint Identification	p.7
	C. Problem Specification	p.8
	D. Solution Generation	p.9
	E. Solution Evaluation	p.10
	F. Safety/Professionalism	p.10
	G. Detailed Design	p.11
	H. Solution Assessment	p.17
IV.	Live-Long Learning	p.18
V.	Conclusions	p.19
VI.	References	p.20
VII.	Bibliography	p.20
VIII.	Appendices	p.20

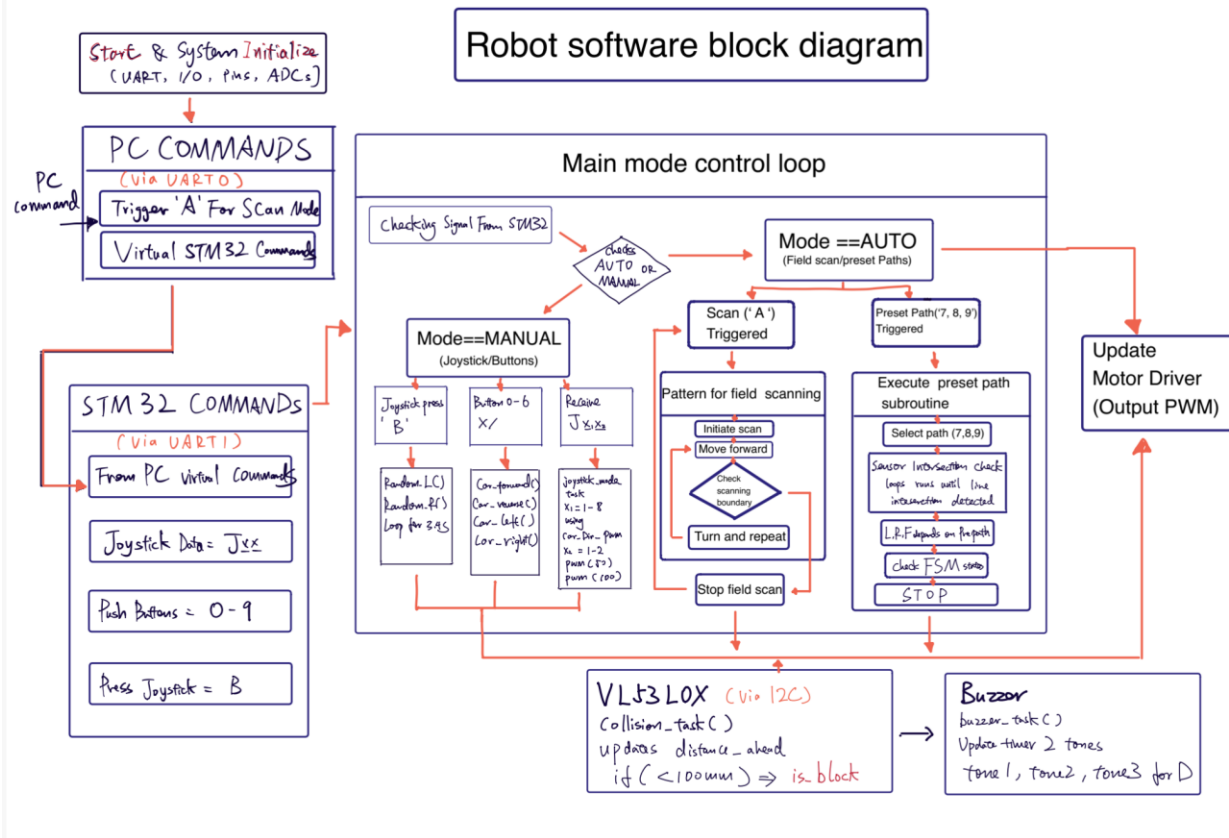
I. Introduction

This project uses embedded C programming to design, build, and test a remote controlled autonomous robot that is able to follow a guided wire. The robot is powered by an EFM8LB12 microcontroller, batteries, and MOSFET drivers, and operates in manual, auto, and mapping modes. In manual mode, an STM32-based remote sends IR (infrared) commands to control the robot's movement; in auto mode, the robot follows the guided wire by detecting its magnetic field with inductors, while its behaviour at intersections is determined by a pre-programmed set of turn instructions. The robot's movement can be controlled by 10 pushbuttons and a joystick on the physical remote, or via a UART user interface running on a Python GUI or a local Bluetooth server. Our design also includes collision detection: the robot slows down and emits short beeps when an obstacle is detected within a range of 20cm, and stops with a constant beep if it's within 10cm. Lastly, in mapping mode, the robot measures the magnetic field in an area and generates a Python plot which approximates the shape of the guided wire track.

Overview of Design Approach



Hardware design figure (refer to Appendix A)



Software Design System Block figure (refer to Appendix A)

II. Investigation

A. Idea Generation

At the start of this project, our team focused on implementing a feasible robot design that could follow the guided wire, recognize intersections, and switch between manual, auto, (and later, mapping) modes. For wire tracking, we considered how sensors should be placed on the chassis to give reliable position information relative to the wire. We decided to investigate a three-inductor arrangement as this could provide separate left and right tracking information while also having a dedicated middle sensor for intersection detection in auto-mode.

For robot control, we investigated how commands could be transmitted from the remote controller to the robot. Since the project required a remote-controlled manual mode, we selected

IR (infrared) communication as a simple and effective method. This led to the use of ASCII commands sent by an SFH45466 IR transmitter and received by a TSOP33338 receiver. These commands were intended not only for manual movements but also for selecting pre-programmed auto-mode paths and, later, starting the mapping mode.

For the microcontrollers, we selected the STM32L051 for the remote controller and the EFM8LB12F64 for the robot. The STM32 incorporates a “high-performance Arm Cortex-M0+ 32-bit RISC core” operating at 32 MHz and an “extensive range of enhanced I/Os,” [2] making it well-suited for button inputs, UART support, and IR transmission. The EFM8 features a “powerful 8051 core with 72 MHz performance and a 14-bit, 20-ch. ADC,” [1] providing fast control response and strong analog sensing capability for the inductors.

B. Design Investigation

After completing the first prototype and initial testing, our team identified several critical performance issues, including missed intersection detections, drifting off track after turns, and inaccurate 180° rotations. To investigate the wire tracking and intersection detection problems, we repeatedly monitored the ADC readings of all three inductors through PuTTY while adjusting the relevant threshold values in the embedded C makefiles. After each adjustment, we ran the robot in auto-mode, observing and recording whether it remained on track and detected every intersection. These tests were repeated until the robot completed three consecutive tests with no errors. Furthermore, to correct the 180° turn, we consistently fine-tuned the fixed duration in the EFM8 makefile (Appendix B) and observed the resulting rotation angle. This continued until the rotation angle was acceptably close to 180°.

C. Data Collection

To debug the wire-tracking system, our team recorded the ADC readings produced by the robot's left, middle, and right inductors. These readings were observed through PuTTY and used to compare the robot's sensor response under different operating conditions, including different robot positions relative to the wire and different sections of the track. An example of these ADC readings is provided in Appendix E. In addition to sensor readings, we recorded the robot's observable performance during each auto-mode test, including the number of intersections missed and whether it drifted away from or completely left the guided wire after turning. These observations allowed us to compare the robot's performance after each threshold adjustment.

Regarding the 180° turn, we observed and recorded the robot's angle of rotation every time after changing the loop count parameter i in the makefile. We also monitored battery voltage during testing, having noticed that changes in motor supply voltage affected the robot's speed, intersection detection, and overall track following. Together, these measurements and observations formed the main body of data used in fine-tuning and evaluation.

D. Data Synthesis

After recording ADC readings and performance observations from repeated auto-mode tests, our team analyzed the results to identify patterns that helped improve the robot's control logic. For track following, we used the robot's routine to use three levels of correction, along with an extra-large "curve" error for much larger adjustments. This allowed the robot to respond differently depending on the sensor readings and apply stronger corrections when necessary. Similarly, for intersection detection, by comparing the middle inductor's ADC readings during normal tracking mode and at intersections, we selected a threshold that reliably distinguished these two conditions, ensuring our robot's consistent performance in auto mode. The recorded

data and observations were, in general, effectively used to determine the threshold values, levels of correction, and timing parameters used in our final design.

E. Analysis of Results

With the final threshold values and timing parameters in Appendix B, the robot was able to follow the guided wire reliably at a reasonable speed, consistently detect intersections, and execute the correct actions for the selected auto-mode path. The 180° turn was also improved, having seen the robot rotating acceptably close to 180° during the latest test. These show that our fine-tuning process significantly improved the consistency of our robot's overall performance.

One interesting observation during this process was that the robot's performance was strongly affected by battery voltage. When the motor supply voltage dropped, the robot not only moved more slowly but also became more likely to miss intersections or drift off track. We also observed that the inductors' ADC readings became less reliable under these circumstances, which reduced the effectiveness of the previously chosen threshold values, demanding further debugging and re-tuning of these values. This indicates that the robot's performance depended not only on the control logic itself but also on maintaining a stable power supply.

As a result, although our final adjustments were valid under normal testing conditions, they were limited by the sensitivity of the system to the battery's condition. To reduce this source of error during testing, we ensured that the 9V motor supply remained close to full charge, and prepared spare batteries to change into whenever performance began to degrade. To conclude, the final robot design performed well, yet its dependence on battery voltage remains an important limitation to our current system.

III. Design

A. Use of Process

Our group followed an iterative design process in which the robot and remote were first divided into smaller functional blocks, then tested individually before being merged into the final system. The robot side used the EFM8LB12F64, and the remote used the STM32L051, satisfying the requirement for two microcontrollers from different families. We developed the motor control, infrared communication, autonomous tracking, collision detection, buzzer, and path handling modules separately, then integrated them through the main program in a single makefile so that all features could run together in a structured way. This approach reduced debugging difficulty and made it easier to improve one subsystem without affecting the others.

As testing progressed, the design was refined through repeated hardware and software adjustments. For example, once basic movement and command reception worked, we added path selection, joystick support, and scanner-based testing to improve functionality and verification. This step-by-step process was crucial because the project needed to reliably and collaboratively perform multiple sensing and control tasks within the constraints of a small embedded system.

B. Needs and Constraints Identification

The project required a robot that could operate in both manual and autonomous modes. To meet user needs, the design should be cost-efficient, easy to assemble and repair, and scalable. To meet enterprise needs, it should also be easily reproducible. In manual mode, it had to receive infrared commands from a remote controller and perform actions such as stop, forward, backward, left, right, and 180° rotation. In auto mode, it had to follow the magnetic field of the guide wire, detect intersections, execute pre-configured path decisions, and stop when an obstacle was detected in front of it. These needs defined the core functions that our final system had to achieve.

Several constraints also shaped the design. The project had to be implemented in C, use two microcontrollers from different families, be battery powered, and use discrete MOSFET motor drivers. In addition, the IR communication had to work through a 38 kHz receiver system, while collision detection had to use the VL53L0X sensor through I2C. The operating range between the controller and the receiver was also an important constraint, since the receiver could only detect commands reliably when the transmitted infrared signal was strong enough. If the controller was too far from the robot, the signal could become too weak or inconsistent, which would reduce communication reliability. Battery operation introduced another practical constraint because the supply voltage was highest when the batteries were new and gradually decreased as the robot operated. This voltage change affected motor behaviour and sensor readings, meaning values such as driving speed, turning speed, and ADC detection thresholds could not always remain perfectly constant. As a result, the system had to be designed such that it would still operate reliably despite battery voltage decay, limited communication range, limited I/O pins, limited RAM, and the need to remain responsive during sensing and movement.

C. Problem Specification

At a minimum, the system had to support manual infrared control, autonomous wire following, pre-configured path selection at intersections, and obstacle detection while remaining reliable in real operation. This required the robot to combine sensing, communication, and motor control in a way that allowed it to react correctly in real time. In addition, the remote side had to provide an effective way for the user to issue commands and change modes without making the system overly complicated.

Beyond the required functions, we also wanted the system to be easier to control, easier to test, and easier to demonstrate. For that reason, the problem expanded to include several

additional features: a distance-based buzzer warning that changes as the robot approaches an obstacle, joystick control on the remote, a laptop-based UI that replicates the physical controller functions, a local server that allows robot control from a phone, a scanner that moves back and forth to measure magnetic field intensity, and a Python-based 2D field intensity plot generated from the recorded data. Therefore, the final problem was not only to make the robot work, but also to improve usability, feedback, testing capability, and presentation of the system.

D. Solution Generation

To meet the core project requirements, we developed a robot controller system with separate functional modules for motion control, communication, autonomous tracking, and collision detection. This modular approach made it easier to build, test, and refine each function before combining everything into the final system. For obstacle handling, the basic requirement was simply to stop or avoid an object, but we improved this by adding a buzzer that begins warning when an obstacle is within 200 mm, increases its beep rate as the obstacle gets closer, and becomes a continuous tone within 100 mm. This provided the user with immediate feedback about the robot's surroundings instead of relying only on the robot's stopping behaviour. For manual control, instead of relying only on fixed command buttons, we added a joystick to the remote so that the robot could be controlled more naturally. We also created a laptop UI with the same functions as the physical remote, including both buttons and joystick control, so commands could be entered and verified more conveniently during testing and demonstrations.

We then extended the system further to improve flexibility and analysis of robot performance. A local server was added so the same commands could also be sent from a phone, giving another practical method of control without changing the robot hardware. On the sensing side, we developed a scanner that could move back and forth across the field, record magnetic

field intensity data into a text file, and then use Python to convert that data into a 2D field intensity plot. This allowed us to move beyond simple observation and evaluate the field-following environment more quantitatively. Overall, these additional solutions were developed to overcome the limitations of a basic robot design by improving operator interaction, system feedback, and field visualization, thereby making the final project more complete and more effective for testing and evaluation.

E. Solution Evaluation

The final solution was selected because it satisfied the main project requirements while also providing several practical improvements to the overall system. The robot was able to support manual infrared control, autonomous wire following, pre-configured path selection, and obstacle detection, which were the core required functions. The modular structure of the software also made the system easier to integrate and troubleshoot, since motion control, communication, autonomous tracking, and collision detection could be developed and tested separately before being combined. This made the final design more reliable than a less structured approach.

The added features also improved the quality of the final solution. The buzzer provided immediate feedback about nearby obstacles, the joystick made manual control more natural, and the laptop UI and phone-based server gave more flexible ways to send commands to the robot. In addition, the scanner and Python-generated 2D field intensity plot gave a more detailed way to evaluate the magnetic field used for tracking, which was useful for testing and analysis. Overall, the final solution was evaluated as effective because it not only met the required project objectives but also improved usability, feedback, and system evaluation without changing the core design.

F. Safety/Professionalism

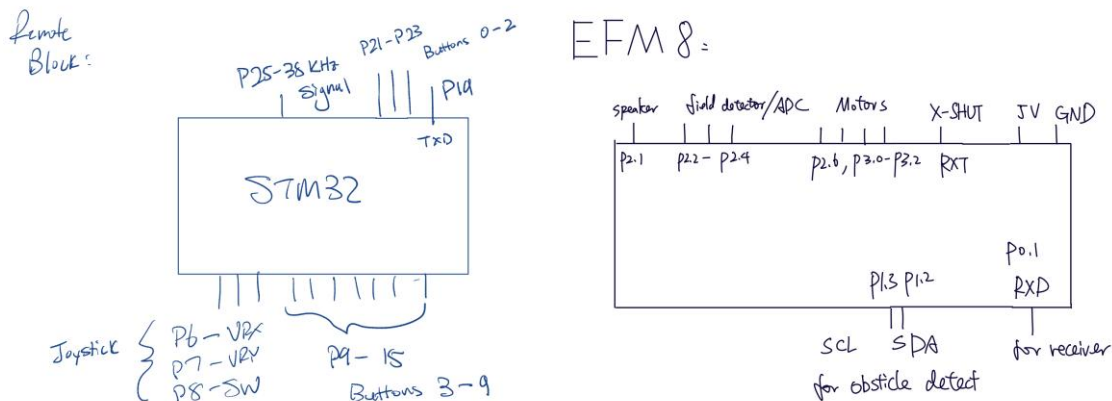
Our team recognizes that we are in a confined space and must behave both safely and professionally. To do this, we've always turned off appliances that are unused, especially the soldering iron. We also need to make sure nothing is being short-circuited at all times, and monitor our circuit for any potential issues. As well, we need to respect others' decisions and communicate well with each other.

G. Detailed Design

Note: all block diagrams are in Appendix A.

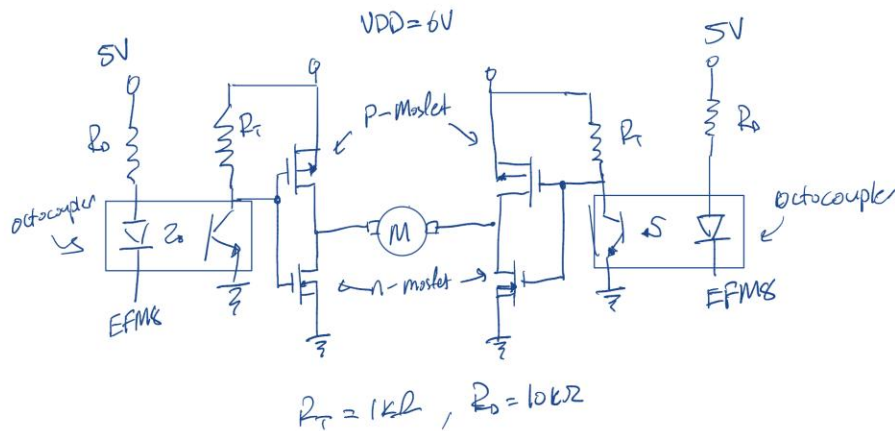
Hardware:

Below are the pinouts for both microcontrollers. The EFM8 was used for the robot while the STM32 was used for the remote control. Attached in Appendix A are the overall hardware wiring block diagrams.



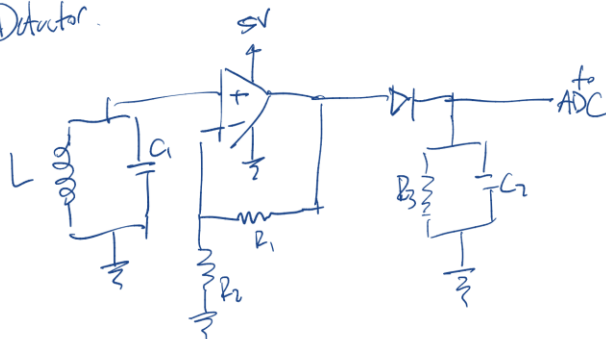
Below are the explanations of all the individual parts. The speaker and buttons were voided as it is too simple. The joystick module will be explained further in the software part.

H - Bridge



To drive the two motors of the robot, we use two H-bridges and an octocoupler to achieve the best result. An optocoupler is a component that uses light inside a chip to send a signal from one circuit to another while keeping them electrically isolated for safety and noise protection. We control the motor then by sending 1's and 0's; sending both 1's or 0's is a stop, and 0 on one side and 1 on the other dictates CCW or CW motion.

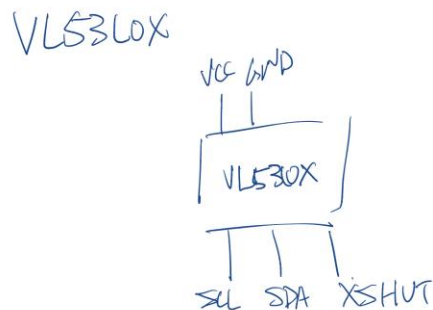
Field Detector.



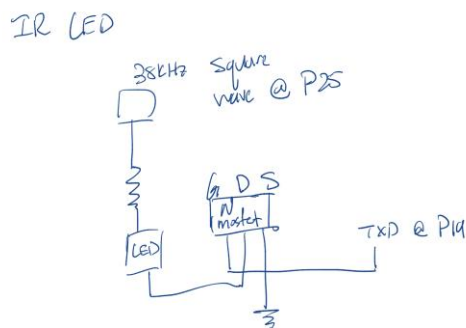
$$\begin{array}{lll} R_1 = 10k\Omega & R_3 = 10k\Omega & C_1 = 0.1\mu F \\ R_2 = 1k\Omega & L = 1mA & C_2 = 0.1\mu F \end{array}$$

This is an LC field-detector front end: the coil L and capacitor C1 form a resonant sensor that picks up a changing magnetic field, and the op-amp amplifies that small signal with gain set

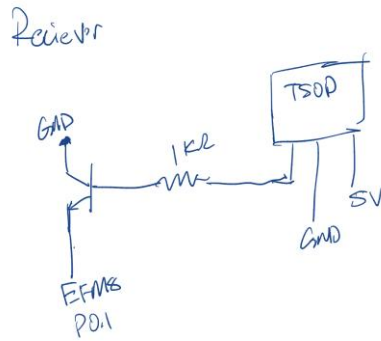
by R1 and R2. The diode then rectifies the amplified AC signal into a DC level, and R3/C2 smooths it so the ADC sees a steady voltage proportional to the detected field strength.



The VL53L0X talks to your microcontroller through SCL and SDA, which are the two lines of the I2C bus. SCL is the clock line controlled by the microcontroller, SDA carries the data back and forth, and XSHUT is connected to a GPIO to turn the sensor off or reset it in software.



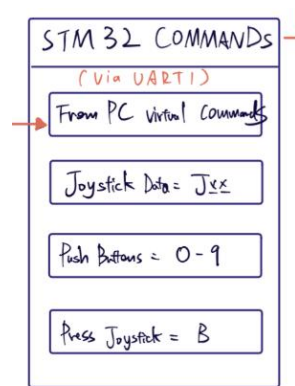
On the STM32 side, we connected the LED and TXD pins to an n-MOSFET in order to stabilize the signal and reduce the error. We used an internally produced 38kHz square wave using Timer 2 to fuel the IR LED. This way, the signal is transmitted through the TXD pins, which send ASCII characters that are then read by the RXD of the receiver.



On the EFM8 side, we noticed that the signal was inverted due to the LED being active low, so we added an NPN inverter and connected the EFM8 to the collector side.

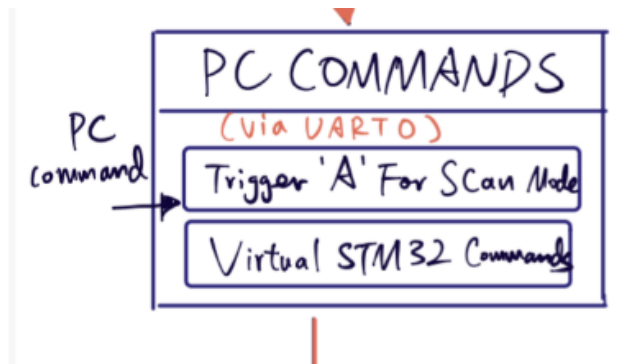
Software

Attached in Appendix A is the overall software block diagram, albeit quite simplified. It will be further broken down into the following blocks.

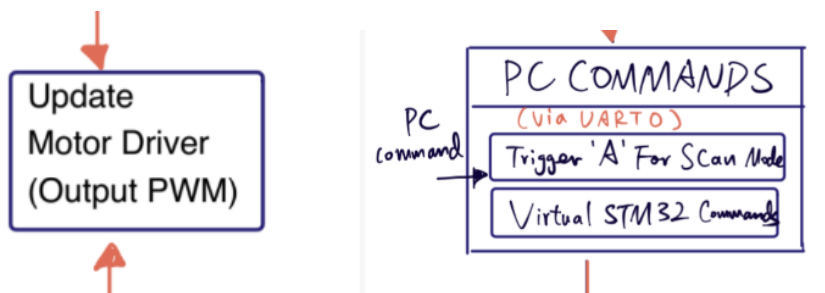


We established communication between the STM32 and the EFM8 using different encodings for each command. First, the buttons are labelled from 0-9, corresponding to stop, forward, backward, left, right, 180° turn, and paths 1-4. The joystick is encoded as JXX, where the first X represents the movement commands (e.g. forward, backward, and the diagonal directions like forward-left and backward-right). The second X represents the speed level, where 1 is slow and 2 is fast. We sent the joystick commands by reading the ADC values of the

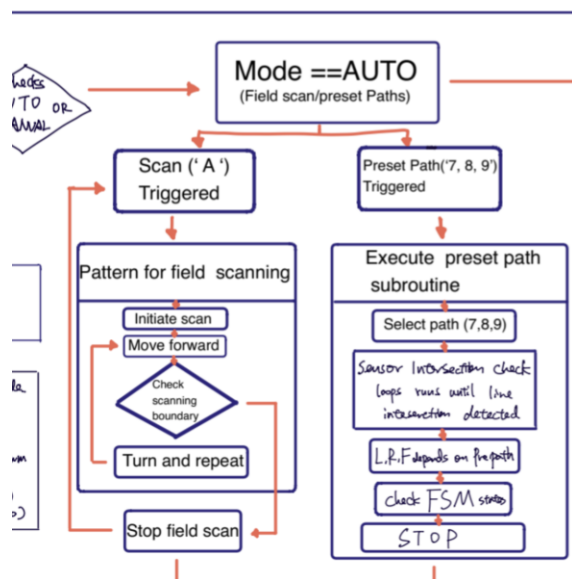
joystick's x and y coordinates and then normalizing them to a range of -100 to 100, where 0 corresponds to stop. Whenever there is a large change in the movement, the command JXX will be sent. In addition, pressing the joystick button sends the breakdance mode command, 'B'.



For the PC commands, it can perform everything the STM32 can send. In addition, it can send 'A' and preset path condition, Path 4. For 'A', it triggers scan mode, in which the robot scans a rectangular area and sends ADC data back to the computer, allowing our Python program to reconstruct the approximate track shape from the collected data. For path 4, we can preconfigure a path on our computer and send it to the robot for it to follow. We implemented this by encoding the command as P.....X, where P marks the beginning of the path and X indicates when the array is complete. We also limited the path to a maximum of 8 commands consisting of left, right, and straight. Lastly, we implemented a local server that allows any device to connect to the computer and control the robot remotely.

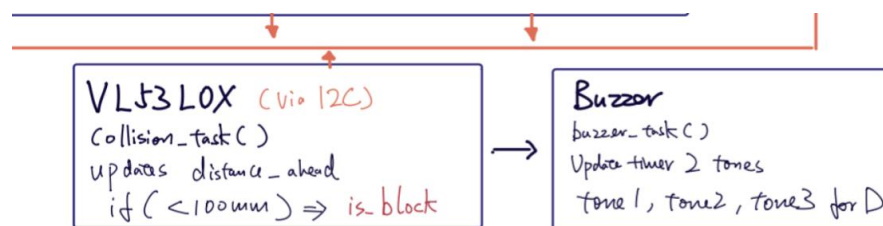


In manual mode, the robot responds directly to commands received from the STM32. These commands determine the requested motion, such as forward, backward, left, right, or stop, making the robot move according to user input rather than following the autonomous path logic. PWM is used to control how much power is delivered to the motors during manual movement. By varying the duty cycle of the pulse signal, the controller changes the average voltage applied to the motors, allowing the robot to adjust speed and turning behaviour while still using digital outputs and efficient switching devices. Furthermore, for the joystick, we included another mode that interprets the encodings and adjusts the speeds to ensure that they are appropriate.



In auto mode, the robot is no longer waiting for direct movement commands from the user; instead, it runs one of its built-in autonomous paths. If scan mode is triggered by the 'A' command, the robot starts a field-scanning routine: it begins scanning (reading ADC values from the 3 inductors), moves forward, checks whether it has reached the scan boundary, then turns and repeats this process until the scan is complete, after which it stops the field scan. If a preset path is triggered (e.g. path 7, 8, or 9), the robot enters the preset-path subroutine. It first selects the

chosen path, then keeps checking for line intersections using its sensors; at each intersection, it follows the stored instruction for that path, such as left, right, forward, or stop. The follow-track routine continuously reads the field sensors and adjusts the robot's motion, ensuring that it always stays aligned with the guide wire. We adjusted the following track module to have 3 different errors and one extra-large "curve" error to fix a much larger amount as well; we also adjusted the values for the middle intersection detection and the duration of turning. This continues while the FSM updates the robot's state, and the routine ends once the final stop instruction is reached. For path 6, FSM reads "P.....X", and then keeps the array stored in memory until 6 is pressed; this array can be overwritten at any time.



The VL53L0X works over the I2C bus and measures the time it takes for the laser to return. At a certain distance before it hits an obstacle, the robot will stop until the obstacle is removed. We also included a buzzer that increases its beeping rate as the robot approaches an obstacle, until the threshold is reached and the robot stops.

H. Solution Assessment

Overall, the robot worked quite well. It successfully completed all of the basic features and also included a few extra features. The graph below demonstrates the tests we performed to balance the motors' forward and backward speeds and optimize their performance. One issue we stumbled upon was the battery, which we also took note of in the graph below, along with how it

affected the results. One limitation with the design was that we had to continuously adjust the middle intersection detection value, as well as the left and right inductor thresholds, so that they match the operating conditions appropriately. We also had to adjust the motor speeds accordingly.

Turn	Entry #	Forward Speed (%)	Backward Speed (%)	Battery Usage at 1h (%)	Battery Usage at 2h (%)	Battery Usage at 3h (%)
Left	1	34	35	12	24	37
	2	35	34	13	25	38
	3	33	35	12	24	36
	4	34	33	13	26	39
Right	1	35	34	12	25	37
	2	34	35	13	26	39
	3	35	33	12	24	36
	4	34	34	13	25	38

As well, we also tested the ADC readings for the middle intersection tracking and follow track code. Appendix E shows a table for our readings that we used to calibrate the robot.

IV. Live-Long Learning

A major learning need in this project was gaining practical experience in integrating multiple embedded subsystems into one working design. While previous labs and courses provided background in C programming, microcontrollers, and basic interfacing, this project required us to combine infrared communication, motor control, I2C sensing, autonomous decision-making, and user interfaces into a single real-time system. This type of full-system integration is not something that can be learned only from lectures, so a large part of the project involved independent learning through datasheets, example code, testing, and debugging.

The project also showed how useful prior course material was when applied to a larger design problem. Knowledge from ELEC 291 labs was especially important for microcontroller programming, timers, ADC use, and transistor-based motor interfacing, while the project itself extended those ideas much further. At the same time, we had to teach ourselves new tools and methods, such as creating a laptop-based control UI, running a local server for phone control, recording magnetic field data, and using Python to generate a 2D field intensity plot.

V. Conclusions

In conclusion, our project successfully demonstrated a remote controlled autonomous robot capable of manual control, guided-wire tracking, pre-configured path selection, obstacle detection, and field mapping. By using the EFM8LB12F64 on the robot and the STM32L051 on the remote, we were able to satisfy the requirement of using two microcontrollers from different families while integrating sensing, communication, and motor control into one complete embedded system. Through repeated testing and refinement, we improved important behaviours such as follow-track accuracy, intersection handling, and 180° turning, allowing the robot to perform reliably in both manual and auto modes.

In addition to meeting the main project requirements, we also enhanced the system with several useful extra features, including obstacle warning through a buzzer, a joystick-based remote control, a laptop and phone-based command interface, and a scanner with Python-based 2D field visualization. These additions improved the usability, flexibility, and testability of the final design. Overall, this project took approximately 60 hours of work from our group and gave us valuable experience in embedded system integration, debugging, and iterative design, while showing how careful hardware-software co-development is necessary to build a practical and reliable robotic system.

VI. References

- [1] “EFM8LB12F64E-QFN32.” *Silabs.com*, 2026, <https://www.silabs.com/mcu/8-bit-microcontrollers/efm8-laser-bee/device.efm8lb12f64e-qfn32?utm>
- [2] “STM32L051C6 | Product - STMicroelectronics.” *STMicroelectronics*, 2026, www.st.com/en/microcontrollers-microprocessors/stm32l051c6.html?utm

VII. Bibliography

- Samek, M., “Practical UML Statecharts in C/C++, 2nd Edition,” Newnes, 2008.
- STMicroelectronics, “VL53L0X Datasheet - Time-of-Flight Ranging Sensor,” 2024.
- Vishay Semiconductors, “TSOP331..., TSOP333..., TSOP335.. IR Receiver Modules for Remote Control Systems,” 2025.

VIII. Appendices

<https://drive.google.com/drive/folders/1XkndlPCxp010RHTHIchQthZdSr6lrmhI?usp=sharing>

The above link gives access to all the appendices (Appendix A - F).